Java Applets Notes (NPTEL Exam Oriented)

> **Programming in Java (NPTEL)**
>
> These notes are designed for quick revision before the NPTEL assignments and final exam.

Chapter 1: Introduction to Applets

What is an Applet?

An **Applet** is a small Java program that runs inside a web browser or an Applet Viewer.

Unlike a normal Java application, an applet **does not contain the `main()` method**.

It extends the `Applet` class provided by Java.

```
public class Demo extends Applet
```

Definition (Exam)

Applet: A Java program that executes inside a browser or Applet Viewer and provides graphical user interaction.

Package Required

```
import java.applet.Applet;
```

For GUI components:

```
import java.awt.*;
```

Features of Applets

- Written in Java
- Platform Independent
- Runs inside browser/AppletViewer
- Uses AWT for GUI
- Event Driven
- Does not have main()
- Supports Graphics

Characteristics

- Cannot execute directly using `java ClassName`
- Must extend `Applet`
- Uses life cycle methods
- Controlled by browser/AppletViewer

Advantages

- Platform independent

- Easy to create GUI
- Supports graphics
- Event handling available
- Secure execution environment (Sandbox)

Disadvantages

- Slow loading
- Browser support removed
- Requires JVM
- Limited permissions
- Obsolete technology

Why were Applets removed?

Applets were removed because

- Browser vendors stopped supporting Java plugins.
- Security vulnerabilities.
- Better technologies like HTML5, JavaScript and JavaFX became available.

Java 9: Deprecated

Java 17: Removed

⭐ **NPTEL Tip:** Even though Applets are obsolete, they remain part of many university and NPTEL syllabi.

Application vs Applet

Feature	Java Application	Java Applet
Entry Point	main()	No main()
Runs Using	JVM	Browser/AppletViewer
Starts With	java command	HTML/AppletViewer
Window	Console/GUI	Browser/Applet
Security	Full access	Sandbox restrictions

Important Differences

Application

- Uses `main()`
- Can access local files
- Can terminate using `System.exit()`

Applet

- No `main()`
- Cannot access local files freely
- Controlled by browser

Applet Architecture



Components

HTML File

Contains information about the Applet.

Example

```
<applet code="Demo.class" width="300" height="300">
</applet>
```

Browser

Loads the HTML page.

JVM

Executes bytecode.

Applet

Actual Java program.

Creating First Applet

Program

```
import java.applet.Applet;
import java.awt.Graphics;

public class Demo extends Applet {

    public void paint(Graphics g) {

        g.drawString("Hello NPTEL",50,50);

    }

}
```

HTML File

```
<html>

<body>

<applet code="Demo.class" width="300" height="300">

</applet>

</body>

</html>
```

Compilation

```
javac Demo.java
```

Execution

```
appletviewer Demo.html
```

Output

```
----------------------------

Hello NPTEL

----------------------------
```

How it Works

Step 1

Compile Java file

↓

Step 2

Creates Demo.class

↓

Step 3

HTML loads Demo.class

↓

Step 4

AppletViewer starts JVM

↓

Step 5

paint() executes

↓

Output displayed

Important Points

✔ No `main()` method.

✔ Must extend `Applet`.

✔ Uses `paint(Graphics g)` for output.

✔ GUI output is displayed using Graphics object.

Frequently Asked MCQs

Q1

Which package contains Applet class?

A. java.io

B. java.awt

C. java.applet

D. java.util

Answer: C

Q2

Which method is NOT present in an Applet?

A. init()

B. paint()

C. start()

D. main()

Answer: D

Q3

Which keyword is used to inherit Applet?

A. implements

B. extends

C. inherit

D. using

Answer: B

Q4

Which class is used for drawing?

A. Color

B. Font

C. Graphics

D. Image

Answer: C

Q5

Can an Applet execute using

```
java Demo
```

Answer: No.

It requires Browser or AppletViewer.

Common Errors

Error

```
main method not found
```

Reason

Trying to execute using

```
java Demo
```

Solution

Use

```
appletviewer Demo.html
```

Error

```
UnsupportedClassVersionError
```

Reason

Compiled using Java 17+

Running using Java 8 AppletViewer.

Solution

Compile using JDK 8 or

```
javac --release 8 Demo.java
```

Error

```
Class not found
```

Reason

Wrong class name inside HTML.

Error

```
paint() not executing
```

Reason

Wrong method signature.

Correct

```
public void paint(Graphics g)
```

NPTEL Exam Tips ⭐

✅ Remember:

- Applet has **no main()**
- Must extend **Applet**
- Output is displayed using **paint()**
- Graphics object performs drawing
- HTML file loads the Applet
- Browser/AppletViewer controls execution

Quick Revision

Applet

↓

extends Applet

↓

Compile

↓

Creates .class

↓

HTML loads class

↓

AppletViewer

↓

JVM

↓

paint()

↓

Output

Chapter 2 : Applet Life Cycle ☆ ☆ ☆ ☆ ☆

> **NPTEL Weightage:** Very High

Almost every NPTEL assignment or exam contains at least one question based on the Applet Life Cycle.

What is Applet Life Cycle?

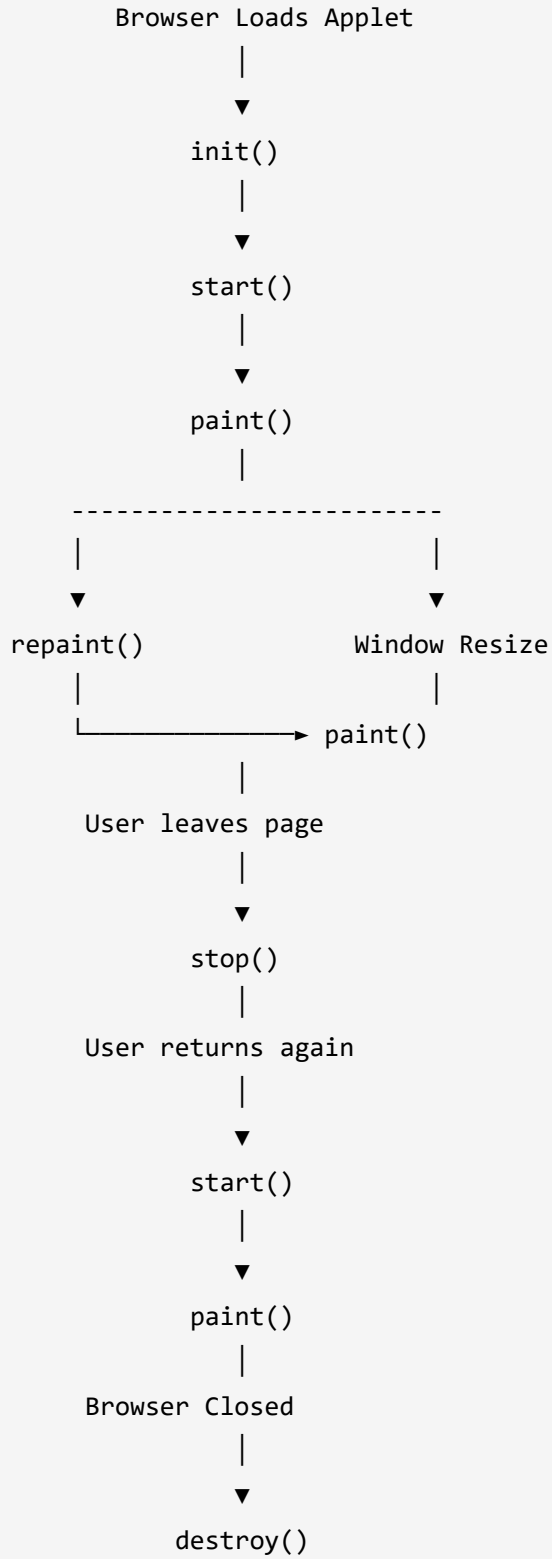
The **Applet Life Cycle** is the sequence of methods automatically called by the browser or AppletViewer during the execution of an applet.

You **never call these methods yourself**. The browser/AppletViewer invokes them automatically.

Life Cycle Methods

Method	Called By	Number of Times	Purpose
init()	Browser	Once	Initialization
start()	Browser	Multiple	Starts or resumes applet
paint()	Browser	Multiple	Draws output
stop()	Browser	Multiple	Stops applet temporarily
destroy()	Browser	Once	Cleanup before removal

Complete Life Cycle



1. init()

Definition

`init()` is called **only once** when the applet is loaded.

Used to initialize variables, objects and GUI components.

Syntax

```
public void init(){

}
```

Uses

- Initialize variables
- Create Buttons
- Create TextFields
- Set Background Color
- Load Images

Example

```
public void init(){

    setBackground(Color.YELLOW);

}
```

Important Points

✔ Executes only once.

✔ Similar to constructor.

✔ Called before start().

Memory Trick

```
init()

↓

Initialize Everything
```

---

2. start()

Definition

Called after init().

Called every time the applet becomes active.

Syntax

```
public void start(){

}
```

Uses

- Resume animation
- Start thread
- Restart timer
- Restart music

Example

```
public void start(){

    System.out.println("Applet Started");

}
```

Important Points

✔ Called many times.

✔ Executes after init().

✔ Executes again when user revisits page.

Memory Trick

```
start()

↓

Resume Execution
```

---

3. paint(Graphics g)

Definition

Responsible for displaying output on screen.

Everything visible on the Applet is drawn using paint().

Syntax

```
public void paint(Graphics g){

}
```

Example

```
public void paint(Graphics g){

    g.drawString("Hello",50,50);

}
```

Important Points

✔ Called automatically.

✔ Can execute many times.

✔ Called after start().

✔ Called whenever screen needs repainting.

---

paint() is called when

- Window opened
- Window resized
- Window restored
- repaint() invoked
- Applet becomes visible

Example

```
public void paint(Graphics g){

    g.drawRect(20,20,100,50);

}
```

---

4. stop()

Definition

Called when the user leaves the web page.

Applet remains in memory but becomes inactive.

Syntax

```
public void stop(){

}
```

Uses

- Stop animations
- Pause thread
- Pause music
- Save temporary data

Important Points

✔ Called multiple times.

✔ Executed before destroy().

Memory Trick

```
User leaves page

↓

stop()
```

---

5. destroy()

Definition

Called once before applet is removed permanently.

Used to release resources.

Syntax

```
public void destroy(){

}
```

Uses

- Close files
- Stop threads
- Release memory
- Close database connection

Important Points

✔ Called only once.

✔ Last method of life cycle.

Memory Trick

```
Browser Closed

↓

destroy()
```

Complete Program

```
import java.applet.Applet;
import java.awt.Graphics;

public class Demo extends Applet{

    public void init(){

        System.out.println("init");

    }

    public void start(){

        System.out.println("start");

    }

    public void paint(Graphics g){

        g.drawString("Applet Running",50,50);

    }

    public void stop(){

        System.out.println("stop");

    }

    public void destroy(){

        System.out.println("destroy");

    }

}
```

Execution Order

```
init()

↓

start()

↓

paint()

↓

stop()

↓

start()

↓

paint()

↓

destroy()
```

Life Cycle Table

Method	Called Once	Called Multiple Times
init()	✔	✖
start()	✖	✔
paint()	✖	✔
stop()	✖	✔
destroy()	✔	✖

---

init() vs start()

init()	start()
Executes once	Executes many times
Initialization	Resume execution
Called first	Called after init()

---

start() vs paint()

start()	paint()
Starts execution	Draws output
No Graphics object	Uses Graphics object

---

stop() vs destroy()

stop()	destroy()
Temporary	Permanent
Called many times	Called once
Pause execution	Cleanup resources

---

paint() vs repaint() ☆ ☆ ☆

One of the favorite NPTEL questions.

paint()	repaint()
Draws output	Requests redraw
Called automatically	Called by programmer
Has Graphics parameter	No parameter

Example

```
repaint();
```

Actually performs

```
repaint()

↓

Browser receives request

↓

paint(Graphics g)
```

Important: `repaint()` does **not** draw anything itself. It simply tells the system that the applet should be redrawn.

Frequently Asked MCQs

Q1

Which method executes only once?

A. start()

B. paint()

C. init()

D. stop()

Answer: C

Q2

Which method is responsible for drawing?

A. destroy()

B. init()

C. paint()

D. start()

Answer: C

Q3

Which method executes when user leaves page?

A. stop()

B. init()

C. destroy()

D. paint()

Answer: A

Q4

Which method is executed last?

A. stop()

B. paint()

C. destroy()

D. start()

Answer: C

Q5

Which method can execute multiple times?

A. init()

B. destroy()

C. start()

D. Both start() and paint()

Answer: D

Q6

Which method resumes execution after the user revisits the page?

A. init()

B. paint()

C. start()

D. destroy()

Answer: C

Q7

Which method has a Graphics parameter?

A. init()

B. start()

C. paint()

D. stop()

Answer: C

NPTEL-style Questions

Q1

Arrange the following methods in the correct execution order:

- paint()
- destroy()
- init()
- start()

Answer

```
init()

↓

start()

↓

paint()

↓

destroy()
```

Q2

State whether the following are True or False.

1. init() executes every time the applet becomes active.
   False
2. destroy() executes only once.
   True
3. paint() may execute multiple times.
   True
4. stop() is called when the applet becomes inactive.
   True
5. repaint() directly draws graphics.
   False

Q3

Fill in the blanks.

1. _____ is responsible for drawing graphics.
   paint()
2. _____ is called only once.
   init()
3. _____ is used to resume execution.
   start()
4. _____ is the last life cycle method.
   destroy()

Common Mistakes

❌ Calling `paint()` directly.

✔ Use

```
repaint();
```

❌ Writing

```
paint()
```

✔ Correct

```
public void paint(Graphics g)
```

❌ Assuming `start()` executes only once.

✔ It may execute many times.

❌ Confusing `stop()` and `destroy()`.

Remember:

```
stop()

↓

Temporary

destroy()

↓

Permanent
```

Quick Revision ✯

```
init()

↓

Called once

↓

Initialization

---------------------

start()

↓

Resume execution

---------------------

paint()

↓

Display Output

---------------------

stop()

↓

Pause Applet

---------------------

destroy()

↓
```

```
Cleanup

--------------------

paint()

↓

Graphics

--------------------

repaint()

↓

Calls paint()
```

Chapter Summary

- `init()` initializes the applet and runs only once.
- `start()` starts or resumes execution and may run multiple times.
- `paint(Graphics g)` is responsible for all drawing.
- `stop()` pauses the applet when it becomes inactive.
- `destroy()` releases resources before the applet is removed.
- `repaint()` requests the system to invoke `paint()`.

Chapter 3 : Graphics Class (java.awt.Graphics) ☆ ☆ ☆ ☆ ☆

> **NPTEL Weightage:** Very High

The **Graphics** class is used to display text, lines, shapes, images and other graphical objects inside an Applet.

Every drawing operation is performed using the **Graphics object**.

What is Graphics Class?

The `Graphics` class belongs to the **java.awt** package.

It provides methods to draw:

- Text
- Lines
- Rectangles
- Circles
- Ovals
- Arcs
- Polygons
- Images

Without the Graphics object, an Applet cannot display anything.

Required Package

```
import java.awt.*;
```

Graphics Object

The Graphics object is automatically supplied by the browser/AppletViewer.

```
public void paint(Graphics g)
```

Here,

```
g
```

is the Graphics object.

You never create it yourself.

Drawing Text

drawString()

Displays text on the screen.

Syntax

```
g.drawString(String,x,y);
```

Parameters

Parameter	Meaning
String	Text to display

Parameter	Meaning
x	X-coordinate
y	Y-coordinate

Example

```
import java.applet.*;
import java.awt.*;

public class Demo extends Applet{

    public void paint(Graphics g){

        g.drawString("Hello NPTEL",100,100);

    }

}
```

Output

```
Hello NPTEL
```

Important Points

✔ Simplest Graphics method

✔ Coordinates specify starting position

✔ Frequently used in exams

Drawing Line

drawLine()

Draws a straight line.

Syntax

```
g.drawLine(x1,y1,x2,y2);
```

Example

```
public void paint(Graphics g){

    g.drawLine(50,50,200,50);

}
```

Output

```
------------------------
```

Parameters

Parameter	Meaning
x1	Starting X
y1	Starting Y
x2	Ending X
y2	Ending Y

Drawing Rectangle

drawRect()

Draws only the boundary.

Syntax

```
g.drawRect(x,y,width,height);
```

Example

```
g.drawRect(50,50,150,100);
```

Output

```
+--------------+
|              |
|              |
+--------------+
```

Parameters

Parameter	Meaning
x	Starting X
y	Starting Y
width	Width
height	Height

Filled Rectangle

fillRect()

Draws a filled rectangle.

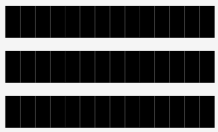
Syntax

```
g.fillRect(x,y,width,height);
```

Example

```
g.fillRect(50,50,150,100);
```

Output



---

drawRect() vs fillRect()

drawRect()	fillRect()
Boundary only	Filled
Hollow	Solid

Drawing Oval

drawOval()

Draws the boundary of an oval.

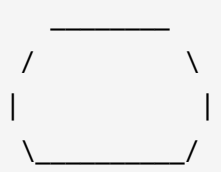
Syntax

```
g.drawOval(x,y,width,height);
```

Example

```
g.drawOval(100,100,150,80);
```

Output



Filled Oval

fillOval()

Draws a filled oval.

Syntax

```
g.fillOval(x,y,width,height);
```

Example

```
g.fillOval(100,100,150,80);
```

Important Point

If

```
width = height
```

then the shape becomes a **Circle**.

Example

```
g.drawOval(100,100,100,100);
```

Produces

```
o
```

---

drawOval() vs fillOval()

drawOval()	fillOval()
Hollow	Filled

Drawing Round Rectangle

drawRoundRect()

Rectangle having rounded corners.

Syntax

```
g.drawRoundRect(x,y,width,height,arcWidth,arcHeight);
```

Example

```
g.drawRoundRect(50,50,200,100,30,30);
```

Filled Round Rectangle

fillRoundRect()

Filled rounded rectangle.

Syntax

```
g.fillRoundRect(x,y,width,height,30,30);
```

Drawing Arc

drawArc()

Draws part of a circle.

Syntax

```
g.drawArc(x,y,width,height,startAngle,arcAngle);
```

Example

```
g.drawArc(100,100,100,100,0,180);
```

Output

```
  _______

 /       \

------------
```

Parameters

Parameter	Meaning
startAngle	Starting angle
arcAngle	Angle covered

Filled Arc

fillArc()

Fills the arc.

Syntax

```
g.fillArc(x,y,width,height,0,180);
```

Polygon

A polygon is a figure having multiple sides.

---

drawPolygon()

Syntax

```
int x[]={50,100,150};

int y[]={150,50,150};

g.drawPolygon(x,y,3);
```

Output

```
   /\
  /  \
 /____\
```

---

fillPolygon()

Syntax

```
g.fillPolygon(x,y,3);
```

Graphics Methods Summary

Method	Purpose
drawString()	Display text
drawLine()	Draw line
drawRect()	Rectangle
fillRect()	Filled rectangle
drawOval()	Oval
fillOval()	Filled oval
drawArc()	Arc
fillArc()	Filled arc
drawRoundRect()	Rounded rectangle

Method	Purpose
fillRoundRect()	Filled rounded rectangle
drawPolygon()	Polygon
fillPolygon()	Filled polygon

Coordinates in Graphics

```
(0,0)
+----------------------------------->

|

|

|

v

Y-axis
```

Top-left corner is

```
(0,0)
```

Increasing X

→ Right

Increasing Y

↓

Downwards

Mini Program

```
import java.applet.*;
import java.awt.*;

public class Shapes extends Applet{

    public void paint(Graphics g){

        g.drawString("Graphics Demo",40,30);

        g.drawLine(20,50,150,50);

        g.drawRect(20,70,100,50);

        g.fillRect(150,70,100,50);

        g.drawOval(20,150,100,80);

        g.fillOval(150,150,100,80);

    }

}
```

Frequently Asked MCQs

Q1

Which method displays text?

A. drawText()

B. print()

C. drawString()

D. write()

Answer: C

Q2

Which method draws a straight line?

A. line()

B. draw()

C. drawLine()

D. printLine()

Answer: C

Q3

Which method creates a filled rectangle?

A. drawRect()

B. rectangle()

C. fillRect()

D. solidRect()

Answer: C

Q4

Which method draws an oval?

A. drawCircle()

B. drawOval()

C. circle()

D. drawRound()

Answer: B

Q5

Which object performs drawing?

A. Applet

B. Graphics

C. Color

D. Font

Answer: B

Q6

Which package contains Graphics?

A. java.io

B. java.awt

C. java.util

D. java.net

Answer: B

Q7

Which method is used to draw a polygon?

A. drawTriangle()

B. drawPolygon()

C. polygon()

D. drawShape()

Answer: B

NPTEL-style Questions

Q1

State True or False.

1. `drawRect()` draws a filled rectangle.

False

2. `fillOval()` creates a solid oval.

True

3. `drawString()` displays text.

True

4. Graphics belongs to `java.awt`.

True

Q2

Fill in the blanks.

1. _____ object performs drawing.

Graphics

2. _____ displays text.

drawString()

3. _____ creates a filled rectangle.

fillRect()

4. _____ draws multiple connected lines.

drawPolygon()

Common Mistakes

❌ Forgetting to import

```
import java.awt.*;
```

❌ Writing

```
drawRect(...)
```

instead of

```
g.drawRect(...)
```

❌ Using

```
paint()
```

instead of

```
public void paint(Graphics g)
```

NPTEL Tips ⭐

✔ Remember the syntax of all Graphics methods.

✔ Know the difference between `draw` and `fill` methods.

✔ `Graphics` object is automatically supplied to `paint()`.

✔ Coordinates start from the **top-left corner (0,0)**.

✔ If width = height in `drawOval()`, it becomes a **circle**.

Quick Revision

```
Graphics Object

↓

drawString()

↓

drawLine()

↓

drawRect()

↓

fillRect()

↓

drawOval()

↓

fillOval()

↓

drawArc()

↓

fillArc()

↓

drawPolygon()

↓
```

```
fillPolygon()
```

Chapter Summary

- The `Graphics` class is used for all drawing operations in an Applet.
- The Graphics object is passed automatically to the `paint(Graphics g)` method.
- `draw...()` methods create outlines, while `fill...()` methods create solid shapes.
- Coordinates begin at the top-left corner `(0,0)` and increase rightward (X) and downward (Y).

Chapter 4 : Colors and Fonts ☆ ☆ ☆ ☆

> **NPTEL Weightage:** Medium

The `Graphics` class can draw text and shapes, but they appear in the **default color (black)** and **default font** unless we change them.

Java provides the **Color** and **Font** classes to customize the appearance.

Color Class

The **Color** class belongs to the `java.awt` package.

```
import java.awt.*;
```

Why use Color?

Color is used to

- Change text color
- Change shape color
- Change background color

---

setColor()

Changes the current drawing color.

Syntax

```
g.setColor(Color.RED);
```

After calling `setColor()`, every shape or text drawn will use that color until another color is selected.

Example

```
public void paint(Graphics g){

    g.setColor(Color.RED);

    g.drawString("Hello",50,50);

}
```

Output

```
Hello

(Color = RED)
```

Predefined Colors

Java provides many predefined colors.

Color Constant	Output Color
Color.BLACK	Black
Color.BLUE	Blue
Color.RED	Red
Color.GREEN	Green
Color.YELLOW	Yellow
Color.ORANGE	Orange
Color.PINK	Pink
Color.CYAN	Cyan
Color.MAGENTA	Magenta
Color.GRAY	Gray
Color.WHITE	White

Example

```
g.setColor(Color.BLUE);

g.drawRect(50,50,100,100);

g.setColor(Color.RED);

g.fillOval(200,50,100,100);
```

Creating Custom Colors

We can also create our own colors using RGB values.

RGB means

- Red
- Green
- Blue

Each value ranges from

```
0 to 255
```

Syntax

```
Color c = new Color(red,green,blue);
```

Example

```
Color c = new Color(255,128,0);

g.setColor(c);

g.fillRect(50,50,100,100);
```

RGB Examples

RGB	Color
(255,0,0)	Red
(0,255,0)	Green
(0,0,255)	Blue
(255,255,0)	Yellow
(255,255,255)	White
(0,0,0)	Black

Background Color

The background color of an applet can be changed using

```
setBackground()
```

Syntax

```
setBackground(Color.YELLOW);
```

Usually written inside

```
init()
```

Example

```
public void init(){

    setBackground(Color.YELLOW);

}
```

Foreground Color

Changes the default drawing color.

Syntax

```
setForeground(Color.RED);
```

Complete Color Example

```
import java.applet.*;
import java.awt.*;

public class ColorDemo extends Applet{

    public void init(){

        setBackground(Color.YELLOW);

    }

    public void paint(Graphics g){

        g.setColor(Color.RED);

        g.drawString("Java",50,50);

        g.setColor(Color.BLUE);

        g.drawRect(50,80,100,80);

        g.setColor(Color.GREEN);

        g.fillOval(200,80,100,80);

    }

}
```

Font Class

The Font class changes

- Font Style
- Font Name
- Font Size

Font Constructor

```
Font(String name,int style,int size)
```

Parameters

Parameter	Meaning
name	Font Name
style	Font Style
size	Font Size

Font Styles

Constant	Meaning
Font.PLAIN	Normal
Font.BOLD	Bold
Font.ITALIC	Italic
Font.BOLD+Font.ITALIC	Bold Italic

Creating Font

Example

```
Font f = new Font("Arial",Font.BOLD,20);
```

---

setFont()

Applies the selected font.

Syntax

```
g.setFont(f);
```

Example

```
public void paint(Graphics g){

    Font f = new Font("Arial",Font.BOLD,24);

    g.setFont(f);

    g.drawString("NPTEL",50,50);

}
```

Output

```
NPTEL

(Font = Arial Bold 24)
```

Another Example

```
Font f = new Font("Times New Roman",Font.ITALIC,30);

g.setFont(f);

g.drawString("Java Applet",50,80);
```

Complete Program

```
import java.applet.*;
import java.awt.*;

public class FontDemo extends Applet{

    public void init(){

        setBackground(Color.WHITE);

    }

    public void paint(Graphics g){

        g.setColor(Color.BLUE);

        Font f=new Font("Arial",Font.BOLD,24);

        g.setFont(f);

        g.drawString("Programming in Java",40,60);

    }

}
```

Important Methods

Method	Purpose
setColor()	Changes drawing color
setBackground()	Changes background

Method	Purpose
setForeground()	Changes foreground
setFont()	Changes font

Frequently Asked MCQs

Q1

Which method changes drawing color?

A. setPaint()

B. setColor()

C. setBackground()

D. drawColor()

Answer: B

Q2

Which class represents colors?

A. Graphics

B. Color

C. Paint

D. Brush

Answer: B

Q3

Which package contains Color?

A. java.io

B. java.awt

C. java.util

D. java.net

Answer: B

Q4

Which constructor creates a Font?

A.

```
Font()
```

B.

```
new Font(name,style,size)
```

C.

```
Font.create()
```

D.

```
Font.make()
```

Answer: B

Q5

Which constant represents bold font?

A.

```
Font.BOLD
```

B.

```
Font.THICK
```

C.

```
Font.HEAVY
```

D.

```
Font.DARK
```

Answer: A

Q6

RGB values range between

A. 0-100

B. 0-200

C. 0-255

D. 1-256

Answer: C

Q7

Which method changes the applet background?

A.

```
setBackground()
```

B.

```
setColor()
```

C.

```
setPaint()
```

D.

```
drawBackground()
```

Answer: A

NPTEL-style Questions

Q1

Fill in the blanks.

1.

```
g._____(Color.RED);
```

Answer

```
setColor
```

2.

```
_____ changes the background color.
```

Answer

```
setBackground()
```

3.

```
Font.BOLD
```

represents

Answer

Bold Font

Q2

State True or False.

1. `setColor()` changes the current drawing color.

True

2. RGB values range from 0 to 255.

True

3. `Font.PLAIN` represents italic font.

False

4. `setBackground()` is commonly used inside `init()`.

True

Common Mistakes

❌ Forgetting to apply the font.

Wrong

```
Font f=new Font("Arial",Font.BOLD,20);
```

Correct

```
Font f=new Font("Arial",Font.BOLD,20);

g.setFont(f);
```

❌ Assuming `setColor()` changes only one shape.

Actually,

every drawing after `setColor()` uses that color until another color is selected.

❌ Using invalid RGB values.

Wrong

```
new Color(300,20,10);
```

Correct

```
new Color(255,20,10);
```

Memory Tricks ⭐

Color

```
setColor()

↓

Drawing Color
```

Background

```
setBackground()

↓

Applet Background
```

Font

```
new Font()

↓

Create Font

↓

setFont()

↓

Apply Font
```

Quick Revision

```
Color Class

↓

setColor()

↓

setBackground()

↓

RGB (0-255)

↓

Font Class

↓

new Font()

↓

Font.PLAIN

↓

Font.BOLD

↓

Font.ITALIC

↓

setFont()
```

Chapter Summary

- `Color` class is used to change drawing and background colors.
- `setColor()` changes the current drawing color.
- `setBackground()` changes the applet's background.
- Custom colors can be created using RGB values (0–255).
- `Font` class controls the font name, style, and size.
- `setFont()` applies the selected font before drawing text.

Chapter 5 : Images in Applets ☆ ☆ ☆

> **NPTEL Weightage:** Low to Medium

Images make Applets more interactive. Java provides the **Image** class to load and display images.

Image Class

The **Image** class belongs to the `java.awt` package.

```
import java.awt.*;
```

Why use Images?

Images are used to display

- Logos

- Icons
- Backgrounds
- Animations
- Pictures

Methods Used

Method	Purpose
getImage()	Loads image
drawImage()	Displays image

Loading an Image

An image is generally loaded inside **init()**.

Syntax

```
Image img;

public void init(){

    img = getImage(getDocumentBase(),"flower.jpg");

}
```

Explanation

getDocumentBase()

Returns the location of the HTML file.

flower.jpg

Image file stored in the same folder.

Displaying an Image

After loading the image, display it using

```
drawImage()
```

Syntax

```
g.drawImage(img,x,y,this);
```

Parameters

Parameter	Meaning
img	Image Object
x	X Coordinate

Parameter	Meaning
y	Y Coordinate
this	Current Applet

Complete Program

```
import java.applet.*;
import java.awt.*;

public class ImageDemo extends Applet{

    Image img;

    public void init(){

        img=getImage(getDocumentBase(),"flower.jpg");

    }

    public void paint(Graphics g){

        g.drawImage(img,50,50,this);

    }

}
```

Output

```
+------------------------+

      [ Flower Image ]

+------------------------+
```

Loading Image From URL

```
Image img;

img=getImage(new URL("http://example.com/image.jpg"));
```

> **Note:** Rarely asked in NPTEL.

---

drawImage() Overloaded Methods

Method 1

```
g.drawImage(img,x,y,this);
```

Displays image using original size.

Method 2

```
g.drawImage(img,x,y,width,height,this);
```

Displays image with custom size.

Example

```
g.drawImage(img,50,50,200,150,this);
```

Image Scaling

Original Image

```
100 × 100
```

Displayed As

```
200 × 200
```

using

```
drawImage()
```

Applet Using Text and Image

```
import java.applet.*;
import java.awt.*;

public class Logo extends Applet{

    Image img;

    public void init(){

        img=getImage(getDocumentBase(),"logo.png");

    }

    public void paint(Graphics g){

        g.drawString("Programming in Java",40,40);

        g.drawImage(img,40,70,this);

    }

}
```

Important Points

✔ Image must be loaded before displaying.

✔ Usually loaded in

```
init()
```

✔ Displayed in

```
paint()
```

Image Life Cycle

```
init()

↓

getImage()

↓

Image Loaded

↓

paint()

↓

drawImage()

↓

Displayed
```

Frequently Asked MCQs

Q1

Which class is used to store images?

A. Picture

B. Image

C. Graphics

D. Photo

Answer: B

Q2

Which method loads an image?

A.

```
loadImage()
```

B.

```
getImage()
```

C.

```
drawImage()
```

D.

```
Image.load()
```

Answer: B

Q3

Which method displays an image?

A.

```
showImage()
```

B.

```
paintImage()
```

C.

```
drawImage()
```

D.

```
displayImage()
```

Answer: C

Q4

Where should images normally be loaded?

A.

```
paint()
```

B.

```
start()
```

C.

```
init()
```

D.

```
stop()
```

Answer: C

Q5

Which object is required to display images?

A. Font

B. Color

C. Graphics

D. Applet

Answer: C

NPTEL-style Questions

Q1

Fill in the blanks.

Image is loaded using

```
_____
```

Answer

```
getImage()
```

Q2

Fill in the blanks.

Image is displayed using

```
_________
```

Answer

```
drawImage()
```

Q3

State True or False.

1. `drawImage()` loads an image.

False

2. `getImage()` loads an image.

True

3. Images are usually loaded inside `init()`.

True

4. Images are displayed inside `paint()`.

True

Common Errors

Error 1

```
Image not displayed
```

Reason

Wrong file name.

Example

```
flower.jpg
```

Actual file

```
Flower.jpg
```

Java treats them as different names on many systems.

Error 2

Wrong folder.

Image should usually be placed in the same folder as

```
Demo.class
```

or referenced correctly.

Error 3

Trying to display image before loading.

Wrong

```
paint()

↓

drawImage()

↓

getImage()
```

Correct

```
init()

↓

getImage()

↓

paint()

↓

drawImage()
```

Memory Tricks ✯

```
Image

↓

getImage()

↓

Load

↓

drawImage()

↓

Display
```

Quick Revision

```
Image Class

↓

getImage()

↓

Load

↓

drawImage()

↓

Display

↓

Graphics Object
```

Chapter Summary

- `Image` class stores image objects.
- `getImage()` loads an image into memory.
- `drawImage()` displays the loaded image on the screen.
- Images are generally loaded in `init()` and displayed in `paint()`.
- `Graphics` object is required to draw images.

Previous NPTEL Observation ⭐

Questions on images are **less frequent** compared to:

- Applet Life Cycle
- Graphics
- Event Handling

Read this chapter once for theory and MCQs. Do **not** spend much time memorizing image-related methods.

Chapter 6 : Event Handling ⭐ ⭐ ⭐ ⭐ ⭐

> **NPTEL Weightage:** Very High

This is one of the most important chapters in Applets. Every interactive Applet uses **Event Handling**.

Without Event Handling, an Applet can only display graphics. It cannot respond to user actions like button clicks, mouse movement, or keyboard input.

What is an Event?

An **Event** is an action performed by the user or the system.

Examples

- Button Click
- Mouse Click
- Mouse Drag
- Key Press
- Window Resize

- Scroll Bar Movement

Examples of Events

User Action	Event Generated
Click Button	ActionEvent
Mouse Click	MouseEvent
Mouse Move	MouseMotionEvent
Press Key	KeyEvent
Select Checkbox	ItemEvent
Select Choice	ItemEvent
Scroll Bar	AdjustmentEvent

Event Handling

Definition

Event Handling is the mechanism through which Java responds to user-generated events.

Event Delegation Model ⭐ ⭐ ⭐ ⭐ ⭐

Java uses the **Delegation Event Model**.

This is one of the most frequently asked NPTEL questions.

Components of Event Delegation Model

There are **three components**

```
User

↓

Event Source

↓

Event Object

↓

Listener

↓

Response
```

1. Event Source

The component that generates an event.

Examples

- Button
- TextField
- Checkbox
- List
- Choice

2. Event Object

Stores information about the event.

Examples

```
MouseEvent

ActionEvent

KeyEvent
```

---

3. Event Listener

Receives the event and performs action.

Listener is an **interface**.

Flow of Event Handling

```
User Clicks Button

↓

Button Generates Event

↓

ActionEvent Object Created

↓

ActionListener Receives Event

↓

actionPerformed()

↓

Output
```

Event Classes

Event	Generated By
ActionEvent	Button
MouseEvent	Mouse
KeyEvent	Keyboard
ItemEvent	Checkbox, Choice

Event	Generated By
AdjustmentEvent	Scrollbar

Listener Interfaces

Listener	Method
ActionListener	actionPerformed()
MouseListener	mouseClicked() etc.
MouseMotionListener	mouseDragged(), mouseMoved()
KeyListener	keyPressed(), keyReleased(), keyTyped()
ItemListener	itemStateChanged()

ActionEvent

Generated when

- Button clicked
- Menu selected

ActionListener

```
import java.awt.event.*;

public class Demo extends Applet
implements ActionListener
{

}
```

Method

```
public void actionPerformed(ActionEvent e){

}
```

Complete Example

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

public class Demo extends Applet
implements ActionListener{

    Button b;

    public void init(){

        b=new Button("Click");

        add(b);

        b.addActionListener(this);

    }

    public void actionPerformed(ActionEvent e){

        System.out.println("Button Clicked");

    }

}
```

Explanation

```
Create Button

↓

Register Listener

↓

User Click

↓

ActionEvent

↓

actionPerformed()

↓

Output
```

MouseEvent

Generated whenever mouse performs an action.

Examples

- Click
- Press
- Release
- Enter

- Exit

MouseListener

Methods

```
mouseClicked()

mousePressed()

mouseReleased()

mouseEntered()

mouseExited()
```

Example

```
import java.awt.event.*;

public class Demo extends Applet
implements MouseListener{

    public void init(){

        addMouseListener(this);

    }

    public void mouseClicked(MouseEvent e){

        System.out.println("Mouse Clicked");

    }

    public void mousePressed(MouseEvent e){}

    public void mouseReleased(MouseEvent e){}

    public void mouseEntered(MouseEvent e){}

    public void mouseExited(MouseEvent e){}

}
```

MouseMotionListener

Used for

- Mouse Drag

- Mouse Move

Methods

```
mouseDragged()

mouseMoved()
```

Example

```
public void mouseMoved(MouseEvent e){

    System.out.println("Moving");

}
```

KeyEvent

Generated whenever keyboard key is pressed.

KeyListener

Methods

```
keyPressed()

keyReleased()

keyTyped()
```

Example

```
public void keyPressed(KeyEvent e){

    System.out.println("Key Pressed");

}
```

ItemEvent

Generated by

- Checkbox
- Choice
- List

Listener

```
ItemListener
```

Method

```
itemStateChanged()
```

AdjustmentEvent

Generated by

```
Scrollbar
```

Listener

```
AdjustmentListener
```

Method

```
adjustmentValueChanged()
```

Registration of Listener ⭐ ⭐ ⭐

This is one of the favorite MCQ topics.

Every listener must be registered.

Example

```
button.addActionListener(this);
```

Mouse

```
addMouseListener(this);
```

Keyboard

```
addKeyListener(this);
```

Adapter Classes ⭐ ⭐ ⭐

Sometimes we do not want to implement every method.

Java provides Adapter Classes.

Examples

```
MouseAdapter

KeyAdapter

WindowAdapter
```

Example

Instead of implementing

```
mouseClicked()

mouseEntered()

mouseExited()

mouseReleased()

mousePressed()
```

We can write

```
class Demo extends MouseAdapter{

    public void mouseClicked(MouseEvent e){

    }

}
```

Only one method needs to be overridden.

Event Handling Summary

```
User

↓

Source

↓

Event

↓

Listener

↓

Method Executes
```

Frequently Asked MCQs

Q1

Which model is used for event handling?

A. Sequential Model

B. Delegation Event Model

C. MVC

D. Object Model

Answer

B

Q2

Button click generates

A. MouseEvent

B. ActionEvent

C. KeyEvent

D. ItemEvent

Answer

B

Q3

Mouse click generates

A. ActionEvent

B. KeyEvent

C. MouseEvent

D. WindowEvent

Answer

C

Q4

Keyboard generates

A. ActionEvent

B. KeyEvent

C. MouseEvent

D. ItemEvent

Answer

B

Q5

Which interface handles button clicks?

A. MouseListener

B. ActionListener

C. KeyListener

D. ItemListener

Answer

B

Q6

Which method belongs to ActionListener?

A.

```
actionPerformed()
```

B.

```
mouseClicked()
```

C.

```
keyPressed()
```

D.

```
itemStateChanged()
```

Answer

A

Q7

Which listener handles mouse movement?

A. MouseListener

B. MouseMotionListener

C. ActionListener

D. KeyListener

Answer

B

Q8

Which class helps avoid implementing all MouseListener methods?

A.

```
MouseClass
```

B.

```
MouseAdapter
```

C.

```
MouseHelper
```

D.

```
MouseObject
```

Answer

B

Frequently Confused Questions ⭐

MouseListener vs MouseMotionListener

MouseListener	MouseMotionListener
Click	Move
Press	Drag
Release	Move
Enter	
Exit	

ActionEvent vs MouseEvent

ActionEvent	MouseEvent
Button	Mouse
actionPerformed()	mouseClicked()

Listener vs Event

Event	Listener
ActionEvent	ActionListener
MouseEvent	MouseListener
KeyEvent	KeyListener
ItemEvent	ItemListener

NPTEL-style Questions

True/False

1. Java uses Delegation Event Model.

True

---

2. ActionListener handles keyboard events.

False

---

3. MouseAdapter is an Adapter Class.

True

---

4. Button generates ActionEvent.

True

---

5. MouseMotionListener contains mouseMoved().

True

Fill in the blanks

Button Click

↓

Answer

ActionEvent

Mouse Click

↓

Answer

MouseEvent

Keyboard

↓

Answer

KeyEvent

Checkbox

↓

Answer

ItemEvent

Common Mistakes

❌ Forgetting registration

Wrong

```
Button b=new Button("Click");
```

Correct

```
b.addActionListener(this);
```

❌ Confusing

```
ActionEvent

and

ActionListener
```

Remember

Event

↓

Generated

Listener

↓

Handles Event

Memory Tricks ☆ ☆ ☆

```
Button

↓

ActionEvent

↓

ActionListener

↓

actionPerformed()
```

```
Mouse

↓

MouseEvent

↓

MouseListener

↓

mouseClicked()
```

```
Keyboard

↓

KeyEvent

↓

KeyListener

↓

keyPressed()
```

Quick Revision

```
Event

↓

Source

↓

Event Object

↓

Listener

↓

Method

↓

Output
```

Chapter Summary

- An **Event** is an action performed by the user or the system.
- Java follows the **Delegation Event Model**, which consists of **Event Source**, **Event Object**, and **Event Listener**.
- Different components generate different event types (e.g., `Button` → `ActionEvent`, `Mouse` → `MouseEvent`).
- Listener interfaces handle events through callback methods like `actionPerformed()`, `mouseClicked()`, and `keyPressed()`.

- Adapter classes simplify event handling by allowing you to override only the methods you need.

⭐ NPTEL Exam Focus (Must Memorize)

1. Delegation Event Model
2. Event Source, Event Object, Event Listener
3. ActionEvent ↔ ActionListener ↔ actionPerformed()
4. MouseEvent ↔ MouseListener ↔ mouseClicked()
5. KeyEvent ↔ KeyListener ↔ keyPressed()
6. Difference between MouseListener and MouseMotionListener
7. Adapter Classes (MouseAdapter, KeyAdapter)

Chapter 7 : AWT Controls ⭐ ⭐ ⭐ ⭐ ⭐

> **NPTEL Weightage:** Very High

AWT (Abstract Window Toolkit) provides various GUI components called **Controls**.

Controls allow users to interact with an Applet.

Examples

- Button
- Label
- TextField
- TextArea
- Checkbox
- CheckboxGroup
- Choice
- List
- Scrollbar

AWT Package

```
import java.awt.*;
```

Control Creation Steps

Almost every AWT control follows the same steps.

```
Create Object

↓

Customize (Optional)

↓

Add to Applet

↓

Register Listener (if required)
```

Example

```
Button b;

public void init(){

    b=new Button("Click");

    add(b);

}
```

---

1. Button ☆ ☆ ☆ ☆ ☆

A Button is used to perform an action when clicked.

Constructor

```
Button b=new Button("Click Me");
```

Example

```
import java.applet.*;
import java.awt.*;

public class Demo extends Applet{

    Button b;

    public void init(){

        b=new Button("Submit");

        add(b);

    }

}
```

Output

```
-----------------

[ Submit ]

-----------------
```

Important Methods

```
setLabel()

getLabel()

addActionListener()
```

Button Click Event

```
b.addActionListener(this);
```

---

2. Label ☆ ☆ ☆

A Label displays non-editable text.

Constructor

```
Label l=new Label("Welcome");
```

Example

```
Label l;

public void init(){

    l=new Label("Programming in Java");

    add(l);

}
```

Output

```
Programming in Java
```

Methods

```
setText()

getText()

setAlignment()
```

Label Alignments

```
Label.LEFT

Label.RIGHT

Label.CENTER
```

---

3. TextField ☆ ☆ ☆ ☆ ☆

Used to accept **single-line** input.

Constructor

```
TextField t=new TextField(20);
```

OR

```
TextField t=new TextField("Java");
```

Example

```
TextField t;

public void init(){

    t=new TextField(20);

    add(t);

}
```

Output

```
----------------------

|                    |

----------------------
```

Important Methods

```
getText()

setText()

setEditable()

setEchoChar()
```

Password Example

```
t.setEchoChar('*');
```

Output

```
******
```

---

4. TextArea ☆ ☆ ☆

Used for **multiple-line** input.

Constructor

```
TextArea t=new TextArea(5,20);
```

Example

```
TextArea ta;

public void init(){

    ta=new TextArea(5,20);

    add(ta);

}
```

Output

```
--------------------

|

|

|

--------------------
```

Difference

TextField	TextArea
Single Line	Multiple Lines
Small Input	Large Input

---

5. Checkbox ⭐ ⭐ ⭐ ⭐ ⭐

Allows multiple selections.

Constructor

```
Checkbox c=new Checkbox("Java");
```

Example

```
Checkbox c1,c2;

public void init(){

    c1=new Checkbox("Java");

    c2=new Checkbox("Python");

    add(c1);

    add(c2);

}
```

Output

```
☐ Java

☐ Python
```

Methods

```
getState()

setState()

addItemListener()
```

6. CheckboxGroup ☆ ☆ ☆ ☆ ☆

Used to create **Radio Buttons**.

Only **one option** can be selected.

Example

```
CheckboxGroup cg=new CheckboxGroup();

Checkbox c1=new Checkbox("Male",cg,false);

Checkbox c2=new Checkbox("Female",cg,false);

add(c1);

add(c2);
```

Output

```
( ) Male

( ) Female
```

Only one option can be selected.

Checkbox vs CheckboxGroup

☆ ☆ ☆ ☆ ☆

Checkbox	CheckboxGroup
Multiple Selection	Single Selection
Independent	Radio Buttons

---

7. Choice ☆ ☆ ☆ ☆

Creates a **drop-down list**.

Constructor

```
Choice c=new Choice();
```

Example

```
Choice c;

public void init(){

    c=new Choice();

    c.add("Java");

    c.add("Python");

    c.add("C++");

    add(c);

}
```

Output

```
▼ Java

  Python

  C++
```

Methods

```
add()

getSelectedItem()

getSelectedIndex()
```

---

8. List ☆ ☆ ☆

Displays multiple items.

Unlike Choice, it displays several items at once.

Constructor

```
List l=new List();
```

Example

```
List l=new List();

l.add("Java");

l.add("Python");

l.add("C++");

add(l);
```

Output

```
Java

Python

C++
```

Multiple Selection

```
List l=new List(5,true);
```

true

↓

Allows multiple selection.

Choice vs List ☆ ☆ ☆ ☆

Choice	List
Dropdown	List Box
One item visible	Many items visible
Single selection	Single/Multiple

---

9. Scrollbar ☆ ☆ ☆

Used to select a value using a slider.

Constructor

```
Scrollbar s=new Scrollbar();
```

Example

```
Scrollbar s;

public void init(){

    s=new Scrollbar();

    add(s);

}
```

Output

```
--------------------

|====O==========|

--------------------
```

Listener

```
AdjustmentListener
```

Method

```
adjustmentValueChanged()
```

Complete Summary

Control	Purpose	Listener
Button	Click	ActionListener
Label	Display Text	None
TextField	Single Line	ActionListener
TextArea	Multi Line	TextListener
Checkbox	Multiple Selection	ItemListener
CheckboxGroup	Radio Button	ItemListener
Choice	Dropdown	ItemListener
List	List Box	ItemListener
Scrollbar	Slider	AdjustmentListener

Frequently Asked MCQs

Q1

Which control accepts single-line input?

A. TextArea

B. TextField

C. Label

D. List

Answer

B

Q2

Which control accepts multiple-line input?

A. Button

B. Label

C. TextArea

D. Choice

Answer

C

Q3

Which control behaves like a radio button?

A. Checkbox

B. CheckboxGroup

C. List

D. Choice

Answer

B

Q4

Which control allows multiple selection?

A. Checkbox

B. CheckboxGroup

C. Label

D. Button

Answer

A

Q5

Which control creates a drop-down menu?

A. List

B. Choice

C. Button

D. TextArea

Answer

B

Q6

Which control displays text only?

A. Button

B. Label

C. Choice

D. TextField

Answer

B

Q7

Which listener is associated with Scrollbar?

A. ActionListener

B. ItemListener

C. AdjustmentListener

D. MouseListener

Answer

C

NPTEL-style Questions

True/False

1. TextField accepts multiple lines.

False

---

2. CheckboxGroup allows only one selection.

True

---

3. Choice displays a drop-down list.

True

---

4. Button generates ActionEvent.

True

---

5. Label accepts keyboard input.

False

Fill in the blanks

Single-line input

↓

Answer

TextField

Multiple-line input

↓

Answer

TextArea

Drop-down

↓

Answer

Choice

Radio Button

↓

Answer

CheckboxGroup

Common Mistakes

❌ Confusing

Checkbox

and

CheckboxGroup

Remember

Checkbox

↓

Multiple Selection

CheckboxGroup

↓

Single Selection

❌ Confusing

Choice

and

List

Choice

↓

Dropdown

List

↓

List Box

❌ Forgetting

```
add(control);
```

If you don't call `add()`, the control will not appear in the Applet.

Memory Tricks ⭐ ⭐ ⭐

```
Button

↓

Click

↓

ActionEvent
```

```
Label

↓

Display Only
```

```
TextField

↓

One Line
```

```
TextArea

↓

Many Lines
```

```
Checkbox

↓

Many Choices
```

```
CheckboxGroup

↓

One Choice
```

```
Choice

↓

Dropdown
```

```
List

↓

List Box
```

```
Scrollbar

↓

Slider
```

Quick Revision

```
Button

↓

Label

↓

TextField

↓

TextArea

↓

Checkbox

↓

CheckboxGroup

↓

Choice

↓

List

↓

Scrollbar
```

Chapter Summary

- AWT Controls provide graphical user interface components.
- **Button** generates an `ActionEvent`.
- **Label** displays read-only text.
- **TextField** accepts single-line input, while **TextArea** accepts multiple lines.
- **Checkbox** allows multiple selections, whereas **CheckboxGroup** allows only one selection (radio buttons).
- **Choice** creates a drop-down list, while **List** displays a list box.
- **Scrollbar** is used to select values using a slider.

⭐ NPTEL Exam Focus (Must Memorize)

1. TextField vs TextArea
2. Checkbox vs CheckboxGroup
3. Choice vs List
4. Button → ActionEvent → ActionListener
5. Scrollbar → AdjustmentListener
6. Constructors of commonly used controls
7. `getText()`, `setText()`, `getState()`, `getSelectedItem()`

Chapter 8 : Layout Managers ⭐ ⭐ ⭐ ⭐ ⭐

> **NPTEL Weightage:** High

Layout Managers automatically arrange GUI components (Buttons, Labels, TextFields, etc.) inside an Applet or Frame.

Without a Layout Manager, components may overlap or appear in incorrect positions.

What is a Layout Manager?

A **Layout Manager** is an object that controls the size and position of components inside a container.

Container examples

- Applet
- Frame
- Panel
- Dialog

Why are Layout Managers Required?

Without Layout Manager

```
Button1
        Button2

             Button3
```

Components may overlap.

With Layout Manager

```
Button1

Button2

Button3
```

Everything is arranged automatically.

Types of Layout Managers

Layout Manager	Arrangement
FlowLayout	Left to Right
BorderLayout	Five Regions
GridLayout	Rows & Columns
CardLayout	One Component at a Time
GridBagLayout	Flexible Grid

---

1. FlowLayout ☆ ☆ ☆ ☆ ☆

Default Layout Manager for an Applet.

Components are arranged

```
Left → Right
```

When one row is full,

they automatically move to the next row.

Constructor

```
FlowLayout f = new FlowLayout();
```

Example

```
import java.applet.*;
import java.awt.*;

public class Demo extends Applet{

    Button b1,b2,b3;

    public void init(){

        setLayout(new FlowLayout());

        b1=new Button("One");
        b2=new Button("Two");
        b3=new Button("Three");

        add(b1);
        add(b2);
        add(b3);

    }

}
```

Output

```
+---------------------------------+

[One] [Two] [Three]

+---------------------------------+
```

Alignment

Left

```
new FlowLayout(FlowLayout.LEFT);
```

Center

```
new FlowLayout(FlowLayout.CENTER);
```

Right

```
new FlowLayout(FlowLayout.RIGHT);
```

Important Points

✔ Default Applet Layout

✔ Left to Right

✔ Simple

✔ Most commonly used

---

2. BorderLayout ☆ ☆ ☆ ☆ ☆

Container divided into **five regions**.

```
-------------------------
|        NORTH          |
-------------------------
|WEST| CENTER | EAST    |
-------------------------
|       SOUTH           |
-------------------------
```

Regions

- NORTH
- SOUTH
- EAST
- WEST
- CENTER

Constructor

```
BorderLayout b = new BorderLayout();
```

Example

```
setLayout(new BorderLayout());

add(new Button("North"),BorderLayout.NORTH);

add(new Button("South"),BorderLayout.SOUTH);

add(new Button("East"),BorderLayout.EAST);

add(new Button("West"),BorderLayout.WEST);

add(new Button("Center"),BorderLayout.CENTER);
```

Output

```
--------------------------

North

West Center East

South

--------------------------
```

Important Points

✔ Maximum five components

✔ Center occupies remaining space

✔ Frequently asked in MCQs

---

3. GridLayout ☆ ☆ ☆ ☆ ☆

Arranges components in

```
Rows × Columns
```

Every cell has the same size.

Constructor

```
GridLayout(int rows,int cols)
```

Example

```
setLayout(new GridLayout(2,2));

add(new Button("A"));

add(new Button("B"));

add(new Button("C"));

add(new Button("D"));
```

Output

```
-----------------

| A | B |

| C | D |

-----------------
```

Another Example

```
new GridLayout(3,3);
```

Creates

```
1 2 3

4 5 6

7 8 9
```

Important Points

✔ Equal-sized cells

✔ Useful for calculators

✔ Useful for chess board

---

4. CardLayout ☆ ☆ ☆

Displays

```
One Component At A Time
```

Works like pages of a book.

Example

```
Card 1

↓

Card 2

↓

Card 3
```

Only one card is visible.

Constructor

```
CardLayout c = new CardLayout();
```

Methods

```
next()

previous()

first()

last()

show()
```

Applications

- Login Screen
- Wizard
- Forms
- Menu System

---

5. GridBagLayout ⭐ ⭐

Most powerful Layout Manager.

Allows

- Different component sizes
- Different positions
- Flexible arrangement

Constructor

```
GridBagLayout g = new GridBagLayout();
```

Used With

```
GridBagConstraints
```

Example

```
GridBagLayout g=new GridBagLayout();

GridBagConstraints gbc=new GridBagConstraints();
```

Important Points

✔ Most Flexible

✔ Most Complex

✔ Rarely asked in programming questions

Comparison Table ☆ ☆ ☆ ☆ ☆

Feature	Flow	Border	Grid	Card	GridBag
Default Applet Layout	✔	✖	✖	✖	✖
Left to Right	✔	✖	✖	✖	✖
Rows & Columns	✖	✖	✔	✖	✔
Five Regions	✖	✔	✖	✖	✖
One Component Visible	✖	✖	✖	✔	✖
Flexible	✖	Medium	Medium	Medium	✔

Which Layout to Use?

Situation	Layout
Simple Form	FlowLayout
Window with Header/Footer	BorderLayout
Calculator	GridLayout
Login Pages	CardLayout
Complex GUI	GridBagLayout

Frequently Asked MCQs

Q1

Default Layout of an Applet is

A. GridLayout

B. BorderLayout

C. FlowLayout

D. CardLayout

Answer

C

Q2

Which Layout has five regions?

A. GridLayout

B. BorderLayout

C. CardLayout

D. FlowLayout

Answer

B

Q3

Which Layout arranges components in rows and columns?

A. FlowLayout

B. BorderLayout

C. GridLayout

D. CardLayout

Answer

C

Q4

Which Layout displays one component at a time?

A. GridLayout

B. FlowLayout

C. CardLayout

D. BorderLayout

Answer

C

Q5

Which Layout is most flexible?

A. GridLayout

B. BorderLayout

C. FlowLayout

D. GridBagLayout

Answer

D

Q6

Which Layout Manager is easiest?

A. FlowLayout

B. GridBagLayout

C. CardLayout

D. BorderLayout

Answer

A

NPTEL-style Questions

True / False

1. FlowLayout is the default layout of an Applet.

True

---

2. BorderLayout has four regions.

False

(It has five regions.)

---

3. GridLayout creates equal-sized cells.

True

---

4. CardLayout displays all components simultaneously.

False

---

5. GridBagLayout is the most flexible layout.

True

Fill in the blanks

Default Applet Layout

↓

Answer

FlowLayout

Five Regions

↓

Answer

BorderLayout

Rows & Columns

↓

Answer

GridLayout

One Component Visible

↓

Answer

CardLayout

Most Flexible

↓

Answer

GridBagLayout

Common Mistakes

❌ Confusing

FlowLayout

and

GridLayout

Remember

FlowLayout

↓

One after another

GridLayout

↓

Table

❌ Forgetting

```
setLayout(...)
```

Wrong

```
add(button);
```

Correct

```
setLayout(new FlowLayout());

add(button);
```

❌ Thinking BorderLayout supports unlimited components.

It has **five standard regions**.

Memory Tricks ☆ ☆ ☆

```
Flow

↓

Flows
```

```
Border

↓

Five Regions
```

```
Grid

↓

Table
```

```
Card

↓

One Page
```

```
GridBag

↓

Most Powerful
```

Quick Revision

```
FlowLayout

↓

Default

↓

BorderLayout

↓

5 Regions

↓

GridLayout

↓

Rows & Columns

↓

CardLayout

↓

One Card

↓

GridBagLayout

↓

Flexible
```

Chapter Summary

- A **Layout Manager** controls the arrangement of components in a container.
- **FlowLayout** is the default layout for Applets and arranges components left to right.
- **BorderLayout** divides the container into five regions: North, South, East, West, and Center.
- **GridLayout** arranges components in equal-sized rows and columns.
- **CardLayout** displays only one component (card) at a time.
- **GridBagLayout** is the most flexible and is used for complex GUI designs.

⭐ NPTEL Exam Focus (Must Memorize)

1. Default Layout of Applet → **FlowLayout**
2. BorderLayout → **North, South, East, West, Center**
3. GridLayout → **Rows × Columns**
4. CardLayout → **One component at a time**
5. GridBagLayout → **Most Flexible**
6. Comparison between all Layout Managers

Chapter 9 : Important Applet Programs

⭐ ⭐ ⭐ ⭐ ⭐

> **NPTEL Weightage:** Very High

This chapter contains the **most important Applet programs** asked in NPTEL assignments and university examinations.

Exam Tip ⭐

You do **not** need to memorize every line of code.

Remember

- Which Graphics method is used.
- Which Event Listener is required.
- Which AWT controls are used.
- Logic of the program.

Program 1 : Hello World ☆ ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Hello extends Applet{

    public void paint(Graphics g){

        g.drawString("Hello World",100,100);

    }

}
```

Used Methods

```
drawString()
paint()
```

Program 2 : Draw Line ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Line extends Applet{

    public void paint(Graphics g){

        g.drawLine(50,50,200,50);

    }

}
```

Method Used

```
drawLine()
```

Program 3 : Draw Rectangle ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Rectangle extends Applet{

    public void paint(Graphics g){

        g.drawRect(50,50,150,100);

    }

}
```

Program 4 : Filled Rectangle ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class FillRect extends Applet{

    public void paint(Graphics g){

        g.fillRect(50,50,150,100);

    }

}
```

Program 5 : Circle ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Circle extends Applet{

    public void paint(Graphics g){

        g.drawOval(100,100,100,100);

    }

}
```

Important

```
Width = Height

↓

Circle
```

Program 6 : Filled Circle ☆ ☆ ☆

```java
import java.applet.*;
import java.awt.*;

public class FillCircle extends Applet{

    public void paint(Graphics g){

        g.fillOval(100,100,100,100);

    }

}
```

Program 7 : Smiley Face ☆ ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Smiley extends Applet{

    public void paint(Graphics g){

        g.drawOval(100,50,150,150);

        g.fillOval(135,90,15,15);

        g.fillOval(200,90,15,15);

        g.drawArc(140,120,70,50,180,180);

    }

}
```

Methods Used

```
drawOval()

fillOval()

drawArc()
```

Program 8 : Traffic Signal ☆ ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Signal extends Applet{

    public void paint(Graphics g){

        g.drawRect(100,50,100,250);

        g.setColor(Color.RED);

        g.fillOval(120,70,60,60);

        g.setColor(Color.YELLOW);

        g.fillOval(120,140,60,60);

        g.setColor(Color.GREEN);

        g.fillOval(120,210,60,60);

    }

}
```

Program 9 : Indian Flag ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Flag extends Applet{

    public void paint(Graphics g){

        g.setColor(Color.ORANGE);

        g.fillRect(50,50,200,30);

        g.setColor(Color.WHITE);

        g.fillRect(50,80,200,30);

        g.setColor(Color.GREEN);

        g.fillRect(50,110,200,30);

        g.setColor(Color.BLUE);

        g.drawOval(140,83,20,20);

    }

}
```

Program 10 : Chess Board ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Chess extends Applet{

    public void paint(Graphics g){

        int size=40;

        for(int i=0;i<8;i++){

            for(int j=0;j<8;j++){

                if((i+j)%2==0)

                    g.fillRect(j*size,i*size,size,size);

            }

        }

    }

}
```

Program 11 : Concentric Circles ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class CircleDemo extends Applet{

    public void paint(Graphics g){

        for(int i=0;i<5;i++){

            g.drawOval(50+i*20,50+i*20,200-i*40,200-i*40);

        }

    }

}
```

Program 12 : Display Mouse Coordinates ☆ ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

public class MouseDemo extends Applet implements MouseListener{

    int x,y;

    public void init(){

        addMouseListener(this);

    }

    public void mouseClicked(MouseEvent e){

        x=e.getX();

        y=e.getY();

        repaint();

    }

    public void paint(Graphics g){

        g.drawString("X="+x+" Y="+y,50,50);

    }

    public void mousePressed(MouseEvent e){}

    public void mouseReleased(MouseEvent e){}

    public void mouseEntered(MouseEvent e){}

    public void mouseExited(MouseEvent e){}

}
```

Program 13 : Button Click ☆ ☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

public class ButtonDemo extends Applet
implements ActionListener{

    Button b;

    String msg="";

    public void init(){

        b=new Button("Click");

        add(b);

        b.addActionListener(this);

    }

    public void actionPerformed(ActionEvent e){

        msg="Button Clicked";

        repaint();

    }

    public void paint(Graphics g){

        g.drawString(msg,50,80);

    }

}
```

Program 14 : Add Two Numbers

☆ ☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

public class Add extends Applet
implements ActionListener{

    TextField t1,t2,t3;

    Button b;

    public void init(){

        t1=new TextField(5);

        t2=new TextField(5);

        t3=new TextField(5);

        b=new Button("Add");

        add(t1);

        add(t2);

        add(b);

        add(t3);

        b.addActionListener(this);

    }

    public void actionPerformed(ActionEvent e){

        int a=Integer.parseInt(t1.getText());

        int b=Integer.parseInt(t2.getText());
```

```
        t3.setText(""+(a+b));

    }

}
```

Program 15 : Change Background Color ⭐ ⭐ ⭐

```
import java.applet.*;
import java.awt.*;

public class Background extends Applet{

    public void init(){

        setBackground(Color.YELLOW);

    }

}
```

Program 16 : Draw Multiple Shapes

☆ ☆ ☆

```
import java.applet.*;
import java.awt.*;

public class Shapes extends Applet{

    public void paint(Graphics g){

        g.drawRect(20,20,80,60);

        g.fillOval(150,20,80,60);

        g.drawLine(20,120,200,120);

        g.drawString("Graphics",50,160);

    }

}
```

Most Important Graphics Methods

Method	Used For
drawString()	Text
drawLine()	Line
drawRect()	Rectangle
fillRect()	Filled Rectangle

Method	Used For
drawOval()	Circle/Oval
fillOval()	Filled Circle
drawArc()	Smile
drawPolygon()	Polygon

Programs Frequently Asked ⭐ ⭐ ⭐ ⭐ ⭐

If you have less time, prepare these first.

Program	Priority
Hello World	⭐ ⭐ ⭐ ⭐ ⭐
Rectangle	⭐ ⭐ ⭐ ⭐ ⭐
Circle	⭐ ⭐ ⭐ ⭐ ⭐
Smiley Face	⭐ ⭐ ⭐ ⭐
Traffic Signal	⭐ ⭐ ⭐ ⭐
Button Click	⭐ ⭐ ⭐ ⭐ ⭐
Mouse Click	⭐ ⭐ ⭐ ⭐ ⭐
Add Two Numbers	⭐ ⭐ ⭐ ⭐ ⭐
Chess Board	⭐ ⭐ ⭐
Indian Flag	⭐ ⭐ ⭐

Common Viva Questions

Q1

Which method displays text?

Answer

```
drawString()
```

Q2

Which method draws a filled circle?

Answer

```
fillOval()
```

Q3

Which method is used to redraw the Applet?

Answer

```
repaint()
```

Q4

Which method handles button click?

Answer

```
actionPerformed()
```

Q5

Which listener handles mouse click?

Answer

```
MouseListener
```

NPTEL-style MCQs

Q1

Which method draws a smile in a smiley face?

A. drawOval()

B. drawArc()

C. drawLine()

D. drawPolygon()

Answer

B

Q2

Which event is generated when a button is clicked?

A. MouseEvent

B. ActionEvent

C. KeyEvent

D. ItemEvent

Answer

B

Q3

Which method obtains text from a TextField?

A.

```
readText()
```

B.

```
getText()
```

C.

```
input()
```

D.

```
fetchText()
```

Answer

B

Q4

Which method displays the result in a TextField?

A.

```
setText()
```

B.

```
print()
```

C.

```
display()
```

D.

```
show()
```

Answer

A

⭐ Last-Minute Revision

Remember these methods:

```
drawString()

drawLine()

drawRect()

fillRect()

drawOval()

fillOval()

drawArc()

drawPolygon()

setColor()

setBackground()

setFont()

repaint()
```

Remember these listeners:

```
ActionListener

MouseListener

MouseMotionListener

KeyListener

ItemListener

AdjustmentListener
```

Remember these controls:

```
Button

Label

TextField

TextArea

Checkbox

CheckboxGroup

Choice

List

Scrollbar
```

Chapter Summary

For the NPTEL exam, you are **not expected to memorize dozens of complete programs**. Instead:

- Understand the structure of an Applet (`init()`, `paint()`, listeners).
- Know which Graphics methods draw which shapes.
- Be comfortable with simple event-handling programs (Button click, Mouse click).
- Practice 8–10 core programs, especially those involving Graphics and AWT controls.

Chapter 10 : NPTEL-Style MCQs (Most Important) ⭐ ⭐ ⭐ ⭐ ⭐

> **Exam Weightage:** Very High

These questions are **not copied from NPTEL assignments**, but they follow the same pattern and cover the concepts that are repeatedly tested.

Section A : Basic Applet

Q1

Which package contains the Applet class?

A. java.io

B. java.util

C. java.applet

D. java.awt

✅ **Answer:** C

Q2

Which class should every Applet extend?

A. JFrame

B. Frame

C. Applet

D. Graphics

✅ **Answer:** C

Q3

Which method is NOT present in an Applet?

A. init()

B. start()

C. paint()

D. main()

✅ **Answer:** D

Q4

An Applet executes inside

A. JVM only

B. Browser/AppletViewer

C. DOS

D. Compiler

✅ **Answer:** B

Q5

Which statement is TRUE?

A. Applet has main()

B. Applet extends Applet class

C. Applet extends JFrame

D. Applet extends Graphics

✅ **Answer:** B

Section B : Life Cycle ⭐ ⭐ ⭐ ⭐ ⭐

Q6

Which method executes only once?

A. paint()

B. stop()

C. init()

D. start()

✅ **Answer:** C

Q7

Which method executes when the applet becomes active?

A. stop()

B. destroy()

C. start()

D. paint()

✅ **Answer:** C

Q8

Which method is responsible for drawing?

A. init()

B. paint()

C. stop()

D. destroy()

✅ **Answer:** B

Q9

Which method is called before the applet is removed?

A. start()

B. stop()

C. destroy()

D. paint()

✅ **Answer:** C

Q10

Which method is called when the user leaves the page?

A. stop()

B. destroy()

C. paint()

D. init()

✅ **Answer:** A

Q11

Correct life cycle order is

A.

```
start → init → paint
```

B.

```
init → start → paint
```

C.

```
paint → init → destroy
```

D.

```
paint → start → stop
```

✅ **Answer:** B

Q12

Which method may execute many times?

A. destroy()

B. init()

C. paint()

D. Constructor

✓ **Answer:** C

Section C : Graphics ★ ★ ★ ★ ★

Q13

Which class performs drawing?

A. Color

B. Font

C. Graphics

D. Shape

✓ **Answer:** C

Q14

Which method displays text?

A. print()

B. println()

C. drawString()

D. display()

✅ **Answer:** C

Q15

Which method draws a straight line?

A. draw()

B. line()

C. drawLine()

D. createLine()

✅ **Answer:** C

Q16

Which method draws a filled rectangle?

A. drawRect()

B. fillRect()

C. rectangle()

D. solidRect()

✅ **Answer:** B

Q17

Which method draws an oval?

A. drawCircle()

B. circle()

C. drawOval()

D. oval()

✅ **Answer:** C

Q18

Which method draws a filled oval?

A. fillOval()

B. drawOval()

C. fillCircle()

D. circleFill()

✅ **Answer:** A

Q19

Which method is used to draw a smile?

A. drawLine()

B. drawArc()

C. drawOval()

D. drawPolygon()

✓ **Answer:** B

Q20

Which method draws multiple connected lines?

A. drawTriangle()

B. drawPolygon()

C. drawShape()

D. drawGraph()

✓ **Answer:** B

Section D : Color & Font ☆ ☆ ☆ ☆

Q21

Which method changes drawing color?

A. setPaint()

B. setColor()

C. setBackground()

D. setBrush()

✓ **Answer:** B

Q22

RGB values range from

A. 0–100

B. 0–255

C. 1–100

D. 10–255

✅ **Answer:** B

Q23

Which method changes background?

A. setColor()

B. setBackground()

C. drawBackground()

D. setPaint()

✅ **Answer:** B

Q24

Which class represents fonts?

A. Graphics

B. Font

C. Character

D. Text

✅ **Answer:** B

Q25

Which constant creates bold text?

A. Font.BOLD

B. Font.HEAVY

C. Font.DARK

D. Font.LARGE

✅ **Answer:** A

Section E : Event Handling ⭐ ⭐ ⭐ ⭐ ⭐

Q26

Java follows

A. MVC

B. Delegation Event Model

C. Client Server

D. Observer

✅ **Answer:** B

Q27

Button generates

A. MouseEvent

B. KeyEvent

C. ActionEvent

D. ItemEvent

✅ **Answer:** C

Q28

Mouse click generates

A. MouseEvent

B. ActionEvent

C. KeyEvent

D. WindowEvent

✅ **Answer:** A

Q29

Keyboard generates

A. MouseEvent

B. ActionEvent

C. KeyEvent

D. ItemEvent

✅ **Answer:** C

Q30

Checkbox generates

A. ItemEvent

B. MouseEvent

C. KeyEvent

D. ActionEvent

✅ **Answer:** A

Q31

Which interface handles button click?

A. MouseListener

B. ActionListener

C. KeyListener

D. WindowListener

✅ **Answer:** B

Q32

Which method belongs to ActionListener?

A. mouseClicked()

B. keyPressed()

C. actionPerformed()

D. paint()

✅ **Answer:** C

Q33

Which interface handles keyboard events?

A. MouseListener

B. ActionListener

C. KeyListener

D. ItemListener

✅ **Answer:** C

Q34

Which interface handles mouse movement?

A. MouseMotionListener

B. MouseListener

C. ActionListener

D. WindowListener

✅ **Answer:** A

Q35

Which adapter is used with MouseListener?

A. MouseClass

B. MouseAdapter

C. MouseSupport

D. MouseObject

✓ **Answer:** B

Section F : AWT Controls ⭐ ⭐ ⭐ ⭐ ⭐

Q36

Single-line input is accepted using

A. TextArea

B. TextField

C. Label

D. List

✓ **Answer:** B

Q37

Multi-line input is accepted using

A. TextArea

B. TextField

C. Label

D. Choice

✅ **Answer:** A

Q38

Drop-down list is created using

A. List

B. Choice

C. Button

D. Label

✅ **Answer:** B

Q39

Radio buttons are created using

A. Checkbox

B. CheckboxGroup

C. Choice

D. List

✅ **Answer:** B

Q40

Scrollbar generates

A. MouseEvent

B. ActionEvent

C. AdjustmentEvent

D. ItemEvent

✓ **Answer:** C

Section G : Layout Managers ☆ ☆ ☆ ☆ ☆

Q41

Default Applet Layout is

A. GridLayout

B. FlowLayout

C. BorderLayout

D. CardLayout

✓ **Answer:** B

Q42

Which layout has five regions?

A. FlowLayout

B. GridLayout

C. BorderLayout

D. CardLayout

✓ **Answer:** C

Q43

Rows and Columns are provided by

A. FlowLayout

B. GridLayout

C. BorderLayout

D. CardLayout

✓ **Answer:** B

Q44

One component at a time

A. FlowLayout

B. GridLayout

C. CardLayout

D. BorderLayout

✓ **Answer:** C

Q45

Most flexible layout

A. FlowLayout

B. GridBagLayout

C. GridLayout

D. BorderLayout

✓ **Answer:** B

Section H : True / False ⭐ ⭐ ⭐

1. Applet contains main().

✗ False

---

2. init() executes only once.

✓ True

---

3. paint() may execute multiple times.

✓ True

---

4. Graphics belongs to java.awt.

✓ True

---

5. Button generates ActionEvent.

✓ True

---

6. TextArea accepts single-line input.

✗ False

---

7. FlowLayout is the default Applet Layout.

✓ True

---

8. BorderLayout has five regions.

✓ True

---

9. CheckboxGroup allows multiple selections.

✗ False

---

10. repaint() directly draws graphics.

✗ False

(It requests the system to call paint().)

Frequently Confused Questions

☆ ☆ ☆ ☆ ☆

Question	Correct Answer
Applet Package	java.applet
Graphics Package	java.awt
Color Package	java.awt
Font Package	java.awt
Default Layout	FlowLayout
Draw Text	drawString()
Draw Circle	drawOval()
Filled Circle	fillOval()
Button Event	ActionEvent
Mouse Event	MouseEvent
Keyboard Event	KeyEvent
Checkbox Event	ItemEvent
Scrollbar Event	AdjustmentEvent
Button Listener	ActionListener
Mouse Listener	MouseListener
Keyboard Listener	KeyListener

Chapter 11 : NPTEL Assignment & Exam Pattern Questions ⭐ ⭐ ⭐ ⭐ ⭐

> **Exam Weightage:** Extremely High

This chapter contains questions based on the pattern commonly seen in NPTEL Programming in Java assignments and final exams. These are **practice questions**, not copied from actual NPTEL assignments.

Section 1 : Fill in the Blanks

Q1

An applet is executed inside a _____.

Answer

```
Browser / AppletViewer
```

Q2

The method called only once during the applet life cycle is _____.

Answer

```
init()
```

Q3

The method responsible for drawing graphics is _____.

Answer

```
paint()
```

Q4

The _____ class is used to draw lines, circles and text.

Answer

```
Graphics
```

Q5

The method used to display text is

```
_____
```

Answer

```
drawString()
```

Q6

The default layout manager of an Applet is

```
_____________
```

Answer

```
FlowLayout
```

Q7

A button click generates

```
_____________
```

Answer

```
ActionEvent
```

Q8

The interface that handles button click is

```
_____________
```

Answer

```
ActionListener
```

Q9

The event generated by a checkbox is

```
_____________
```

Answer

```
ItemEvent
```

Q10

A radio button in AWT is created using

```
_____________
```

Answer

```
CheckboxGroup
```

Section 2 : True / False

Q1

An Applet contains a `main()` method.

❌ False

Q2

`paint()` may execute multiple times.

✅ True

Q3

Graphics belongs to `java.awt`.

✓ True

Q4

`fillRect()` draws only the border of a rectangle.

✗ False

Q5

`drawOval()` can draw both circles and ovals.

✓ True

Q6

A `TextArea` accepts multiple lines of input.

✓ True

Q7

A `Choice` component behaves like a drop-down list.

✓ True

Q8

`GridLayout` arranges components into rows and columns.

✅ True

Q9

`BorderLayout` contains five regions.

✅ True

Q10

`GridBagLayout` is more flexible than `FlowLayout`.

✅ True

Section 3 : Match the Following

Column A	Column B
drawString()	Display Text
drawLine()	Draw Line
fillRect()	Filled Rectangle
drawOval()	Circle / Oval
setColor()	Change Drawing Color
setFont()	Change Font

Column A	Column B
ActionListener	Button
MouseListener	Mouse Click
KeyListener	Keyboard
ItemListener	Checkbox

Section 4 : Output-Based Questions

☆ ☆ ☆ ☆ ☆

Question 1

```
public void paint(Graphics g){

    g.drawString("Java",100,100);

}
```

Output

```
Java
```

Question 2

```
g.drawRect(20,20,100,60);
```

Output?

Answer

A rectangle of width **100** and height **60**.

Question 3

```
g.fillOval(50,50,80,80);
```

Output?

Answer

A filled circle.

Question 4

```
setBackground(Color.YELLOW);
```

Output?

Answer

The applet background becomes **Yellow**.

Question 5

```
Button b=new Button("Submit");

add(b);
```

Output?

Answer

A button labeled **Submit** appears.

Section 5 : Identify the Error

Question 1

```
Button b=new Button("OK");
```

Why is the button not visible?

Answer

Because

```
add(b);
```

is missing.

Question 2

```
Font f=new Font("Arial",Font.BOLD,20);
```

Why isn't the font changing?

Answer

Because

```
g.setFont(f);
```

is missing.

Question 3

```
Color.RED;
```

Why is the color not changing?

Answer

Because

```
g.setColor(Color.RED);
```

was not called.

Question 4

```
Button b=new Button("Click");
```

Button click is not working.

Answer

The listener was not registered.

```
b.addActionListener(this);
```

Section 6 : Short Answer Questions

★ ★ ★ ★

Q1

Differentiate between `init()` and `start()`.

init()	start()
Called once	Called every time applet becomes active
Initialization	Starts execution

Q2

Difference between `paint()` and `repaint()`.

paint()	repaint()
Draws graphics	Requests system to call paint()

Q3

Difference between `drawRect()` and `fillRect()`.

drawRect()	fillRect()
Border only	Filled Rectangle

Q4

Difference between `drawOval()` and `fillOval()`.

drawOval()	fillOval()
Border only	Filled Oval

Q5

Difference between `TextField` and `TextArea`.

TextField	TextArea
Single Line	Multiple Lines

Q6

Difference between `Checkbox` and `CheckboxGroup`.

Checkbox	CheckboxGroup
Multiple Selection	Single Selection

Q7

Difference between `Choice` and `List`.

Choice	List
Drop-down	List Box

Q8

Difference between `FlowLayout` and `GridLayout`.

FlowLayout	GridLayout
Left to Right	Rows & Columns

Section 7 : Predict the Event

Component	Event
Button	ActionEvent
Mouse	MouseEvent

Component	Event
Keyboard	KeyEvent
Checkbox	ItemEvent
Scrollbar	AdjustmentEvent

Section 8 : Predict the Listener

Event	Listener
ActionEvent	ActionListener
MouseEvent	MouseListener
KeyEvent	KeyListener
ItemEvent	ItemListener
AdjustmentEvent	AdjustmentListener

Section 9 : Frequently Asked Theory Questions

Q1

Explain the Applet Life Cycle with a diagram.

Answer

```
init()

↓

start()

↓

paint()

↓

stop()

↓

destroy()
```

Q2

Explain the Delegation Event Model.

Answer

The Delegation Event Model consists of three parts:

- Event Source
- Event Object
- Event Listener

Flow:

```
User Action

↓

Source

↓

Event Object

↓

Listener

↓

Handler Method Executes
```

Q3

What is the purpose of the Graphics class?

Answer

The `Graphics` class is used to draw text, lines, rectangles, circles, ovals, polygons, arcs, and images inside an Applet.

Q4

What are Layout Managers?

Answer

Layout Managers automatically arrange GUI components inside containers like Applets and Frames.

Section 10 : Mini Mock Test

Q1

Default layout of Applet?

A. GridLayout

B. BorderLayout

C. FlowLayout

D. CardLayout

Answer: C

Q2

Which method draws text?

A. print()

B. drawText()

C. drawString()

D. display()

Answer: C

Q3

Which event is generated when a button is clicked?

A. MouseEvent

B. ActionEvent

C. KeyEvent

D. WindowEvent

Answer: B

Q4

Which component accepts multiple-line input?

A. TextField

B. Choice

C. TextArea

D. Label

Answer: C

Q5

Which layout divides the container into five regions?

A. GridLayout

B. BorderLayout

C. FlowLayout

D. CardLayout

Answer: B

Chapter 12 : 5-Minute Ultimate Revision Cheat Sheet ☆ ☆ ☆ ☆ ☆

> **Read this once before every NPTEL assignment or final exam.**
>
> If you remember everything on this sheet, you can answer **90%+ of Applet-related MCQs**.

---

1. Applet Basics

Package

```
import java.applet.*;
```

Graphics Package

```
import java.awt.*;
```

Event Package

```
import java.awt.event.*;
```

---

2. Applet vs Java Application

Applet	Application
Runs inside Browser/AppletViewer	Runs using JVM
No main()	Has main()
Extends Applet	Any class
GUI Based	Console/GUI
Uses HTML	No HTML required

---

3. Applet Life Cycle ☆ ☆ ☆ ☆ ☆

```
init()

↓

start()

↓

paint()

↓

stop()

↓

destroy()
```

Meaning

Method	Purpose	Called
init()	Initialization	Once
start()	Start execution	Multiple Times
paint()	Draw Graphics	Multiple Times
stop()	Applet inactive	Multiple Times
destroy()	Cleanup	Once

Memory Trick

```
I

S

P

S

D

↓

Init

Start

Paint

Stop

Destroy
```

4. Graphics Class ⭐ ⭐ ⭐ ⭐ ⭐

Everything is drawn using

```
Graphics g
```

Most Important Methods

Method	Purpose
drawString()	Text
drawLine()	Line
drawRect()	Rectangle
fillRect()	Filled Rectangle
drawOval()	Circle/Oval
fillOval()	Filled Circle
drawArc()	Arc/Smile
drawPolygon()	Polygon
drawImage()	Image

5. Color Class ⭐ ⭐ ⭐ ⭐

```
g.setColor(Color.RED);
```

Background

```
setBackground(Color.YELLOW);
```

RGB

```
new Color(255,0,0);
```

Range

```
0-255
```

---

6. Font Class ☆ ☆ ☆ ☆

Constructor

```
Font f=new Font("Arial",Font.BOLD,20);
```

Apply

```
g.setFont(f);
```

---

7. Images ☆ ☆ ☆

Load

```
getImage()
```

Display

```
drawImage()
```

---

8. Event Handling ☆ ☆ ☆ ☆ ☆

Java uses

```
Delegation Event Model
```

Components

```
Event Source

↓

Event Object

↓

Event Listener
```

---

9. Event → Listener Mapping ⭐ ⭐ ⭐ ⭐ ⭐

Event	Listener	Method
ActionEvent	ActionListener	actionPerformed()
MouseEvent	MouseListener	mouseClicked()
MouseMotionEvent	MouseMotionListener	mouseMoved(), mouseDragged()
KeyEvent	KeyListener	keyPressed()
ItemEvent	ItemListener	itemStateChanged()
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged()

---

10. AWT Controls ⭐ ⭐ ⭐ ⭐ ⭐

Control	Purpose
Button	Click
Label	Display Text
TextField	Single Line
TextArea	Multiple Lines
Checkbox	Multiple Selection
CheckboxGroup	Radio Button
Choice	Dropdown
List	List Box
Scrollbar	Slider

---

11. Control Differences ☆ ☆ ☆ ☆ ☆

TextField vs TextArea

TextField	TextArea
One Line	Many Lines

Checkbox vs CheckboxGroup

Checkbox	CheckboxGroup
Multiple Selection	Single Selection

Choice vs List

Choice	List
Dropdown	List Box

---

12. Layout Managers ☆ ☆ ☆ ☆ ☆

Layout	Feature
FlowLayout	Default Applet Layout
BorderLayout	Five Regions

Layout	Feature
GridLayout	Rows × Columns
CardLayout	One Component Visible
GridBagLayout	Most Flexible

BorderLayout Regions

```
North

West Center East

South
```

---

13. Most Important Constructors

Button

```
new Button("OK");
```

Label

```
new Label("Java");
```

TextField

```
new TextField(20);
```

TextArea

```
new TextArea(5,20);
```

Checkbox

```
new Checkbox("Java");
```

Choice

```
Choice c=new Choice();
```

List

```
List l=new List();
```

---

14. Frequently Used Methods

Graphics

```
drawString()

drawLine()

drawRect()

fillRect()

drawOval()

fillOval()

drawArc()

drawPolygon()
```

Color

```
setColor()

setBackground()
```

Font

```
setFont()
```

TextField

```
getText()

setText()
```

Checkbox

```
getState()

setState()
```

Choice

```
getSelectedItem()
```

List

```
getSelectedIndex()
```

---

15. repaint() ☆ ☆ ☆ ☆ ☆

```
repaint()

↓

Calls

↓

paint()
```

Remember

```
repaint()

DOES NOT

draw directly.
```

---

16. Adapter Classes ☆ ☆ ☆ ☆

Instead of implementing every method

Use

```
MouseAdapter

KeyAdapter

WindowAdapter
```

---

17. Common Programs

Prepare these first.

✔ Hello World

✔ Draw Line

✔ Draw Rectangle

✔ Draw Circle

✔ Smiley

✔ Traffic Signal

✔ Indian Flag

✔ Button Click

✔ Mouse Coordinates

✔ Add Two Numbers

---

18. Most Important MCQs ⭐ ⭐ ⭐ ⭐ ⭐

Default Layout

```
FlowLayout
```

Button Event

```
ActionEvent
```

Button Listener

```
ActionListener
```

Mouse Event

```
MouseEvent
```

Mouse Listener

```
MouseListener
```

Keyboard Event

```
KeyEvent
```

Keyboard Listener

```
KeyListener
```

Checkbox Event

```
ItemEvent
```

Choice Event

```
ItemEvent
```

Scrollbar Event

```
AdjustmentEvent
```

---

19. Frequently Asked Differences

✔ Applet vs Application

✔ init() vs start()

✔ paint() vs repaint()

✔ drawRect() vs fillRect()

✔ drawOval() vs fillOval()

✔ TextField vs TextArea

✔ Checkbox vs CheckboxGroup

✔ Choice vs List

✔ FlowLayout vs GridLayout

20. One-Page Memory Map ☆ ☆ ☆ ☆ ☆

```
Applet

↓

Life Cycle

↓

init()

↓

start()

↓

paint()

↓

Graphics

↓

drawString()

drawRect()

drawOval()

↓

Color

↓

Font

↓
```

```
Images

↓

Event Handling

↓

ActionEvent

MouseEvent

KeyEvent

↓

AWT Controls

↓

Button

Label

TextField

Checkbox

Choice

↓

Layout Managers

↓

FlowLayout

BorderLayout
```

```
GridLayout
```

---

21. Top 30 Facts to Memorize

1. Applet package → `java.applet`
2. Graphics package → `java.awt`
3. Event package → `java.awt.event`
4. No `main()` in Applet
5. `init()` executes once
6. `start()` executes when active
7. `paint()` draws graphics
8. `destroy()` is called before removal
9. Graphics object is required for drawing
10. `drawString()` displays text
11. `drawLine()` draws line
12. `drawRect()` draws rectangle border
13. `fillRect()` fills rectangle
14. `drawOval()` draws circle/oval
15. `fillOval()` fills circle/oval
16. `drawArc()` draws arcs
17. `setColor()` changes drawing color
18. `setBackground()` changes background
19. `Font.BOLD` creates bold text
20. Java uses Delegation Event Model
21. Button → ActionEvent
22. Mouse → MouseEvent
23. Keyboard → KeyEvent
24. Checkbox → ItemEvent
25. TextField → Single Line
26. TextArea → Multiple Lines
27. CheckboxGroup → Radio Button

28. FlowLayout → Default Layout
29. BorderLayout → Five Regions
30. GridBagLayout → Most Flexible

⭐ Final NPTEL Strategy

If you have **1 day left**:

Revise in this order

1. Applet Life Cycle (15 min)
2. Graphics Methods (20 min)
3. Event Handling (30 min)
4. AWT Controls (25 min)
5. Layout Managers (20 min)
6. Solve MCQs (1–2 hours)

🎯 Final Tip

For the **Programming in Java (NPTEL)** course by **Prof. Debasis Samanta**, Applets are **usually a small part of the syllabus**, but the questions are often straightforward and concept-based. Focus on:

- Life Cycle
- Graphics methods
- Event Handling
- AWT Controls
- Layout Managers
- Common MCQs

Avoid spending too much time memorizing long Applet programs. Understanding the concepts

and practicing MCQs will give you a much better return in the exam.

✅ Applet Notes Completed

Contents Covered:

- Chapter 1: Introduction to Applets
- Chapter 2: Applet Life Cycle
- Chapter 3: Graphics Class
- Chapter 4: Colors and Fonts
- Chapter 5: Images in Applets
- Chapter 6: Event Handling
- Chapter 7: AWT Controls
- Chapter 8: Layout Managers
- Chapter 9: Important Programs
- Chapter 10: NPTEL-style MCQs
- Chapter 11: Assignment & Exam Pattern Questions
- Chapter 12: Ultimate Revision Cheat Sheet

These 12 chapters provide a comprehensive Applet revision guide suitable for NPTEL exam preparation.